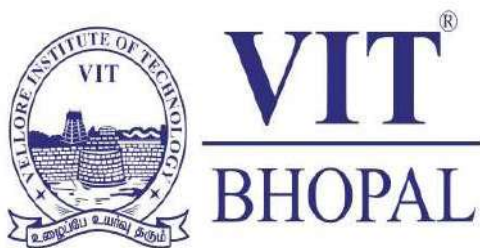


VITYARTHI – PROJECT



Project title: Advanced Number Guessing Game

Course title: Introduction to Problem Solving & Programming

Course code: CSE 1021

Course type: Flipped course

Course credits: 4

Professor: Sivabalan KR

Slot: B21+B22+B23+E14+E21+E22

Submitted by: Leandro Rayan T.M

Registration number: 26MIM10121

Submission date: 08/09/2026

VITYARTHI – PROJECT

INTRODUCTION:

The Advanced Number Guessing Game is an interactive console-based game developed using the Python programming language. The main purpose of the project is to provide a simple but engaging gaming experience in which the computer randomly selects a secret number and the player attempts to identify that number within a limited number of guesses.

The project uses Python's built-in random module to generate a different secret number for every round. The player can select from three difficulty levels: Easy, Medium, and Hard. Each difficulty level provides a different numerical range and a different number of attempts. This makes the game progressively more challenging.

The Easy mode generates a number between 1 and 50 and provides 10 attempts. Medium mode generates a number between 1 and 100 and provides 7 attempts, while Hard mode generates a number between 1 and 200 and provides only 5 attempts.

The program also provides feedback after every valid guess. If the entered number is lower than the secret number, the program displays "Too low!". If the guess is higher, it displays "Too high!". When the player guesses the correct number, the program displays a success message and calculates the score based on the number of attempts used.

An additional dynamic hint is provided after the third unsuccessful attempt. The hint informs the player whether the secret number is even or odd. The program also includes error

VITYARTHI – PROJECT

handling using try-except, ensuring that non-numeric input does not terminate the game.

The game continues through multiple rounds until the user selects the Exit option. The final score is displayed when the player leaves the game.

PROBLEM STATEMENT:

Console-based games are useful for learning programming concepts because they require the programmer to handle user input, decision-making, loops, random values, and program state.

A basic number guessing program can become repetitive if it only asks the user to guess a number without providing different levels of difficulty or meaningful feedback. There is therefore a need for a simple interactive game that combines these fundamental programming concepts into a complete application.

The Advanced Number Guessing Game addresses this problem by providing a structured game environment with different difficulty levels, limited attempts, dynamic feedback, scoring, hints, and error handling.

The system randomly generates a secret number within the selected range. The player then enters guesses until the correct number is found or the maximum number of attempts is reached.

The program must also handle incorrect user input safely. For example, if the user enters letters instead of an integer, the

VITYARTHI – PROJECT

program should display an appropriate error message and allow the player to continue instead of terminating unexpectedly.

The project therefore demonstrates how basic Python programming concepts can be combined to create a complete interactive application.

PROJECT OBJECTIVES:

The major objectives of the Advanced Number Guessing Game are:

1. Random Number Generation

To generate a random secret number within a predefined range using Python's random module.

2. Multiple Difficulty Levels

To provide three difficulty levels:

Easy: 1–50 with 10 attempts

Medium: 1–100 with 7 attempts

Hard: 1–200 with 5 attempts

This allows the player to choose the desired level of challenge.

3. User Interaction

To provide a simple console-based interface through which users can select a difficulty level and enter guesses.

4. Guess Evaluation

VITYARTHI – PROJECT

To compare each user guess with the randomly generated secret number and determine whether the guess is correct, too high, or too low.

5. Attempt Management

To restrict the number of guesses according to the selected difficulty level.

6. Dynamic Hint System

To provide an additional hint after the third failed attempt by identifying whether the secret number is even or odd.

7. Score Management

To maintain a cumulative score across multiple rounds and reward players who identify the number using fewer attempts.

8. Input Validation

To prevent invalid non-integer input from crashing the program by using exception handling.

9. Continuous Gameplay

To allow the user to play multiple rounds without restarting the program.

10. Learning Python Programming

To demonstrate practical use of Python concepts including functions, loops, conditional statements, variables, exception handling, formatted strings, and modules.

FUNCTIONAL REQUIREMENTS :

VITYARTHI – PROJECT

1. Game Initialization

When the program starts, it displays a welcome message:

WELCOME TO ADVANCED GUESS

The score is initialized to zero.

2. Difficulty Selection

The program provides four choices:

Easy

Medium

Hard

Exit Game

The user selects one of these options.

3. Difficulty Configuration

Based on the selected choice, the program assigns a numerical upper limit and maximum number of attempts.

Difficulty, Number Range, Maximum Attempts

Easy, 1–50, 10

Medium, 1–100, 7

Hard, 1–200, 5

4. Random Number Generation

The program generates the secret number using:

`random.randint(1, upper_limit)`

This produces a random integer between 1 and the selected upper limit.

VITYARTHI – PROJECT

5. Guess Input

The user enters a numerical guess during every attempt.

6. Input Validation

The program attempts to convert the entered value into an integer. If conversion fails, a `ValueError` is caught and an error message is displayed.

7. Guess Comparison

The entered guess is compared with the secret number.

If both are equal → Correct guess

If guess is smaller → Too low

If guess is larger → Too high

8. Attempt Tracking

The number of valid guesses is tracked using the attempts variable.

9. Hint Generation

After three unsuccessful attempts, the program checks:

`secret_number % 2`

and determines whether the number is even or odd.

10. Score Calculation

When the player wins, the score is calculated using:

$\text{Score} = (\text{Maximum Attempts} - \text{Attempts Used} + 1) \times 10$

This means that fewer attempts result in a higher score.

11. Multiple Rounds

VITYARTHI – PROJECT

After a round ends, the current total score is displayed and the user can begin another round.

12. Exit

When the user selects option 4, the game displays the final score and terminates.

NON-FUNCTIONAL REQUIREMENTS :

1. Usability

The game should be easy to understand for a beginner. The menu clearly displays the available difficulty levels and the number of attempts.

2. Reliability

The program should continue running even when the user enters invalid non-numeric data.

3. Performance

The game performs only simple calculations and comparisons. Therefore, each operation requires very little processing time.

4. Portability

The project is written in Python and can run on systems with a compatible Python interpreter.

5. Maintainability

The program is organized into a main function called `number_guessing_game()`. The game logic is contained within

VITYARTHI – PROJECT

the function, making the program easier to understand and modify.

6. Resource Efficiency

The application does not require a database, external server, or large external libraries. It mainly uses the built-in random module.

7. Error Tolerance

Invalid numerical input is handled through exception handling instead of allowing the program to terminate.

8. Offline Operation

The game does not require an internet connection. All calculations and random number generation take place locally.

SYSTEM ARCHITECTURE:

The application follows a simple sequential and loop-based architecture.

1. Presentation Layer

This layer communicates directly with the player through the console.

It uses:

print() for displaying messages

input() for receiving user choices and guesses

2. Game Configuration Layer

The selected difficulty determines:

VITYARTHI – PROJECT

Upper numerical limit

Maximum number of attempts

For example:

Easy:

`upper_limit = 50`

`max_attempts = 10`

3. Random Number Generation Layer

The `random.randint()` function generates the secret number.

4. Input Validation Layer

The user's guess is converted into an integer using `int()`.

A try-except block catches `ValueError` if the input cannot be converted into an integer.

5. Game Logic Layer

The program compares the guess and secret number using conditional statements.

6. Hint Layer

After the third failed attempt, the program calculates the parity of the secret number and provides an even/odd hint.

7. Score Management Layer

The score is updated whenever a player successfully guesses the number.

8. Game Control Layer

The outer `while True` loop allows multiple games to be played until the user chooses to exit.

VITYARTHI – PROJECT

DESIGN DIAGRAMS :

1. WORKFLOW DIAGRAM

The overall workflow of the game is:

[Start]



[Display Welcome Message]



[Initialize Score = 0]



[Display Difficulty Menu]



[User Selects Difficulty]



[Validate Menu Choice]



Invalid Choice → Display Error → Return to Menu



Valid Choice



[Set Range and Maximum Attempts]



VITYARTHI – PROJECT

[Generate Random Secret Number]



[Ask User for Guess]



[Validate Guess]



Invalid Input → Display Error → Ask Again



Valid Guess



[Increment Attempts]



[Compare Guess with Secret Number]



Correct?

→ YES → Calculate Score → Display Success → Display Current Score

→ NO → Display Too High / Too Low



Third Failed Attempt?

→ YES → Display Even/Odd Hint



Attempts Remaining?

VITYARTHI – PROJECT

→ YES → Ask for Another Guess

→ NO → Display Correct Number

↓

Return to Difficulty Menu

↓

Exit Selected?

→ YES → Display Final Score → END

DESIGN DECISIONS & RATIONALE :

1. Programming Language: Python

Python was selected because it provides simple syntax, built-in libraries, and convenient input/output functions.

The language is particularly suitable for developing beginner-friendly console applications.

2. Console-Based Interface

A console interface was selected because the primary objective is to demonstrate programming concepts rather than graphical interface development.

It keeps the project lightweight and allows the game logic to remain the main focus.

3. Random Number Generation

The random module was selected to make every round unpredictable.

VITYARTHI – PROJECT

Without randomization, the game would produce the same predictable result.

4. Multiple Difficulty Levels

Three difficulty levels were introduced to increase replay value.

Increasing both the numerical range and reducing the number of attempts creates progressively harder gameplay.

5. Score-Based Reward

A scoring mechanism was included to encourage players to identify the secret number quickly.

The formula rewards successful guesses made with fewer attempts.

6. Dynamic Hint

The even/odd hint is provided after the third failed attempt.

This adds an additional game mechanic without revealing the actual secret number.

7. Exception Handling

The program uses try-except because users may enter invalid values such as letters or symbols.

This makes the game more reliable.

8. Function-Based Design

The complete game is placed inside:

```
number_guessing_game()
```

VITYARTHI – PROJECT

This provides a clear entry point and keeps the program organized.

IMPLEMENTATION DETAILS :

Programming Language

Python

Main Source File

Rayan(1).py

Main Function

number_guessing_game()

Imported Module

random

Main Programming Concepts Used

Variables

Functions

while loops

if-elif-else

User input

Type conversion

Exception handling

Boolean variables

Arithmetic operators

Modulus operator

VITYARTHI – PROJECT

Formatted strings

Python modules

Program Initialization

The program begins with:

```
import random
```

The game function is then defined.

At the bottom of the source file, the condition:

```
if __name__ == "__main__":
```

ensures that the game function is executed when the Python file is run directly.

Difficulty Implementation

The program checks the user's menu choice.

For Easy:

Range = 1–50

Attempts = 10

For Medium:

Range = 1–100

Attempts = 7

For Hard:

Range = 1–200

Attempts = 5

Random Number Implementation

The secret number is generated after difficulty selection.

VITYARTHI – PROJECT

The upper limit depends on the selected difficulty.

Guess Processing

Each valid guess increments the attempts counter.

The program then determines whether the guess is:

Equal to the secret number

Lower than the secret number

Greater than the secret number

Score Implementation

The winning score is based on remaining attempts.

The formula is:

$$\text{Score} = (\text{Maximum Attempts} - \text{Attempts Used} + 1) \times 10$$

The score is added to the cumulative score variable.

TESTING APPROACH:

Testing was performed conceptually across the major functionality implemented in the Python program.

1. Difficulty Selection Testing

The program should correctly recognize:

1 → Easy

2 → Medium

3 → Hard

4 → Exit

An invalid value should result in an error message.

VITYARTHI – PROJECT

2. Random Number Range Testing

The generated number should always remain within the selected range.

For example:

Easy → 1 to 50

Medium → 1 to 100

Hard → 1 to 200

3. Correct Guess Testing

When the user enters the secret number, the game should:

Display a success message

Calculate the score

Set won to True

End the current round

4. Low Guess Testing

When:

$\text{guess} < \text{secret_number}$

the program should display:

Too low!

5. High Guess Testing

When:

$\text{guess} > \text{secret_number}$

the program should display:

Too high!

VITYARTHI – PROJECT

6. Invalid Input Testing

An input such as:

abc

should not terminate the game.

The ValueError exception should be caught and the user should be asked to enter an integer.

7. Attempt Limit Testing

The inner loop should terminate when:

attempts == max_attempts

If the player has not guessed correctly, the program should display the correct number.

8. Hint Testing

After the third unsuccessful valid attempt, the program should calculate whether the secret number is even or odd.

9. Score Testing

The score should increase only after a successful round.

10. Exit Testing

Selecting option 4 should display the final score and terminate the main game loop.

CHALLENGES FACED:

1. Generating Unpredictable Values

VITYARTHI – PROJECT

The game requires a secret number that changes between rounds. This was handled using Python's random module.

2. Managing Multiple Difficulty Levels

Each difficulty requires different ranges and attempt limits.

The program solves this by assigning `upper_limit` and `max_attempts` according to the selected menu option.

3. Handling Invalid Input

Users may enter characters instead of numbers.

Using `try-except` prevents a `ValueError` from stopping the entire application.

4. Managing Multiple Loops

The program requires one loop for repeated games and another loop for attempts within a game.

The outer loop manages the complete game session, while the inner loop manages individual guesses.

5. Maintaining Score Across Rounds

The score must not reset after every round.

Therefore, the score variable is initialized before the game-round loop and updated after successful guesses.

6. Implementing the Hint System

VITYARTHI – PROJECT

The hint must appear only after the third failed valid attempt.

The condition:

if attempts == 3 and not won

ensures that the hint is displayed at the appropriate time.

7. Tracking Winning State

The Boolean variable won is required to distinguish between successful and unsuccessful rounds.

When the player guesses correctly, won becomes True.

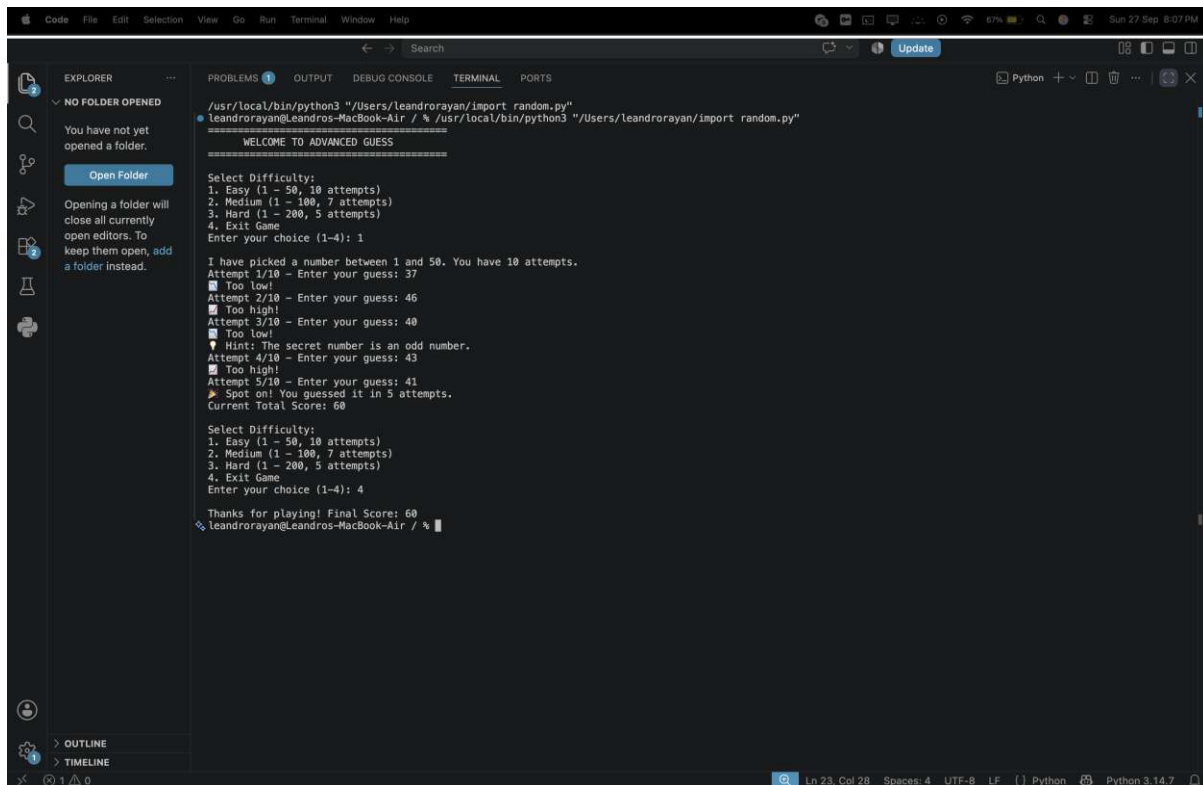
8. Designing a Useful Scoring Formula

The scoring system needs to reward faster guesses.

The implemented formula gives higher points when fewer attempts are used.

OUTPUT:

VITYARTHI – PROJECT



```
Code File Edit Selection View Go Run Terminal Window Help
< > Search
Python + ~ [ ] ... [ ] [ ] [ ]
EXPLORER
NO FOLDER OPENED
You have not yet opened a folder.
Open Folder
Opening a folder will close all currently open editors. To keep them open, add a folder instead.
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/local/bin/python3 "/Users/leandrorayan/import_random.py"
leandrorayan@Leandros-MacBook-Air / % /usr/local/bin/python3 "/Users/leandrorayan/import_random.py"
=====
WELCOME TO ADVANCED GUESS
=====
Select Difficulty:
1. Easy (1 - 50, 10 attempts)
2. Medium (1 - 100, 7 attempts)
3. Hard (1 - 200, 5 attempts)
4. Exit Game
Enter your choice (1-4): 1
I have picked a number between 1 and 50. You have 10 attempts.
Attempt 1/10 - Enter your guess: 37
Too low!
Attempt 2/10 - Enter your guess: 46
Too high!
Attempt 3/10 - Enter your guess: 40
Too low!
Hint: The secret number is an odd number.
Attempt 4/10 - Enter your guess: 43
Too high!
Attempt 5/10 - Enter your guess: 41
Spot on! You guessed it in 5 attempts.
Current Total Score: 60
Select Difficulty:
1. Easy (1 - 50, 10 attempts)
2. Medium (1 - 100, 7 attempts)
3. Hard (1 - 200, 5 attempts)
4. Exit Game
Enter your choice (1-4): 4
Thanks for playing! Final Score: 60
leandrorayan@Leandros-MacBook-Air / %
```

LEARNINGS AND KEY TAKEAWAYS:

1. Use of Python Functions

The project demonstrated how a complete application can be organized inside a function.

2. Randomization

The project provided practical understanding of random number generation using Python's built-in random module.

3. Conditional Logic

The game uses conditional statements extensively for menu selection and guess evaluation.

4. Loop Control

VITYARTHI – PROJECT

The project demonstrated nested loops and how they can be used to manage repeated game rounds and attempts.

5. Exception Handling

The project showed the importance of handling invalid user input through try-except.

6. Boolean State Management

The won variable demonstrates how Boolean values can represent the state of an application.

7. Arithmetic and Modulus Operators

Arithmetic operations are used for score calculation, while the modulus operator is used to determine whether the secret number is even or odd.

8. User Experience

The project demonstrated that simple feedback messages and difficulty selection can make a basic programming exercise more interactive.

9. Problem-Solving

The project required combining several basic Python concepts into one working application.

FUTURE ENHANCEMENTS:

The current version provides the core functionality of an advanced number guessing game. Several improvements can be introduced in future versions.

1. Graphical User Interface

VITYARTHI – PROJECT

A graphical interface could be created using libraries such as Tkinter or PyQt.

2. High Score System

A persistent high-score system could be introduced so that the best score is stored even after the program closes.

3. Player Profiles

Users could enter their names and maintain individual scores.

4. Additional Difficulty Levels

More difficulty levels could be introduced with larger number ranges and fewer attempts.

5. Better Hint System

Additional hints could include:

Range narrowing

Approximate distance from the secret number

Divisibility hints

Multiple-choice clues

6. Timer-Based Gameplay

A countdown timer could be added to introduce another challenge.

7. Persistent Score Storage

Scores could be stored in a text file, CSV file, or database.

8. Sound Effects

Sound effects could be added to improve the gaming experience.

VITYARTHI – PROJECT

9. Multiplayer Mode

A multiplayer version could allow one player to set a number while another player attempts to guess it.

10. Web-Based Version

The game could eventually be converted into a web application using a Python web framework.

REFERENCES:

Python Software Foundation – Python Documentation.

Python Standard Library Documentation – random module.

Python Documentation – Built-in Functions including input(), print(), and int().

Python Documentation – Exception Handling and Value Error.

Python Documentation – Conditional Statements and Loops.